

Test corpus: what we validate against, and what we deliberately do not

The parsers in this repo are only as trustworthy as the input they have met. This records where that input comes from, and one thing we decided **not** to use.

What we use

1. Reference archives from SngCli

`internal/sng/testdata/reference-sngcli-v0.3.0.sng` was produced by **SngCli v0.3.0** (MIT, `mdsitton/SngFileFormat`) from a song folder we wrote. It is the only fixture that checks the reader against something other than our own understanding of the format — the fixtures in `mask_test.go` and `read_test.go` are built by an encoder written from the same reading of the spec as the reader, and two components sharing one misreading agree with each other perfectly.

SngCli is installed at `%LOCALAPPDATA%\Programs\sngcli\win-x64\SngCli.exe`. To regenerate:

```
SngCli.exe encode -i <folder-of-song-folders> -o <out> --noStatusBar -t 1
SngCli.exe decode -i <folder-of-sng-files>    -o <out> --noStatusBar -t 1
```

`.gitignore` excludes `*.sng` so nobody commits a song library, with an explicit exception for `internal/sng/testdata/`.

2. YARG itself, as the end-to-end oracle

YARG v0.15.0 is installed portable at `%LOCALAPPDATA%\Programs\YARG\YARG.exe` — the official Windows x64 zip, no installer, no elevation, deletable in one command. LGPL-3.0-or-later, free, and downloaded from YARC's own release page.

This is the gate that matters for the writer: put a repacked `.sng` in a songs folder, let YARG scan it, and confirm it appears with the right metadata and is playable. Nothing we can assert about our own output substitutes for the client accepting it.

(YARC publishes a `.sig` beside each release asset but no plain checksum, and no public key we could find, so the download is recorded by its SHA-256 rather than verified: `a2eac765a190e34a8acbda5b1351844704b427349d6d36b8cf1096f575055e41`.)

3. Charts we author

Hand-written `song.ini` and chart files exercising the messy cases: BOMs, Latin-1 bytes, duplicate keys, missing section headers, values containing `=`, free-text years, unknown keys, absurd numbers. These are in the unit tests.

What we deliberately do not use

YARG's Official Setlist

It would be the obvious corpus — real charts, real metadata, distributed by YARC themselves — and we are **not** using it.

`YARC-Official/Official-Setlist-Public` states plainly that the setlist is proprietary, that "musicians gave permission to use these songs in YARG specifically", that everything "**must** be used exclusively for YARG", and that users should not download it manually. Using it as the test corpus for a third-party song server is outside that permission — and serving it from one would be squarely outside it.

That is not a close call, and it matters more here than it would elsewhere: this project's own goal is to make charts distributable *separately* from music precisely to avoid copyright problems. A project with that goal cannot be casual about the provenance of its own test data.

The same caution applies to bulk-downloading community charts from Chorus Encore or similar: individual charters' licensing varies, and most community packages bundle commercial audio. Individual charts may be used where the charter's own licence permits it — that is a per-chart question, answered before the chart is added, not a bulk decision.

There IS a documented framework, and it reopens this

The YARN submission guidelines define exactly which music may be used with YARG, which is a better answer than "self-authored only":

- **Acceptable:** CC BY, CC BY-SA, CC BY-NC, CC BY-NC-SA and CC0, at version 3.0 or 4.0; plus the catalogue of accepted royalty-free labels (currently NCS).
- **Not acceptable:** the no-derivatives licences, CC BY-ND and CC BY-NC-ND — because *visual synchronisation is itself a derivative work*. That is a subtlety worth knowing: a chart is a derivative of the song, not merely a companion to it.
- **Share-alike propagates:** a chart of a CC BY-SA song is, by definition, released under the same licence as the song.
- The wiki points at a curated list of Creative Commons songs (the "YARN Planning - CC" tab of the YARN Charts Master Spreadsheet) as a starting point.

So a legitimately-licensed corpus is reachable: a CC BY or CC0 song, charted, is redistributable in full — audio included — and could live in this repository as a real end-to-end fixture rather than a generated one. That is a better long-term answer than the synthetic corpus, and the only reason it is not the current answer is that nobody has charted one yet.

What the oracle actually found, 2026-09-05

`cmd/mkcorpus` wrote 20 deliberately awkward song folders (22 as of the UltraStar work); YARG v0.15.0 scanned them; its `badsongs.txt` and song cache were read back. **Our scanner had passed all 20 and was wrong about three of them.** These are the only findings in this repo that came from an oracle rather than from our own reasoning, which is precisely why they are the ones worth having.

As of the last run, every song YARG rejects is one this scanner independently flags, by three different routes: no parts detected (`13-mid-beats-chart`), `no_audio` (`19-no-audio`), and `ultrastar_no_title` (`21-ultrastar-no-title`). No divergences remain in the corpus.

Case	YARG	We had	Fixed
`song.ini` with no `[Song]` header	reads **nothing**, titles the song after its folder	read the keys	yes — parser now requires the header, and the song is flagged `no_metadata_section`
chart with no audio	rejects: "*No audio accompanying the chart file*"	accepted it silently	yes — flagged `no_audio`
folder with a chart but no `song.ini`	neither cached nor reported bad — silently skipped	accepted it silently	yes — flagged `no_song_ini`

The headerless case is the one that mattered most. Our leniency was a guess written into a comment ("some charts omit it"), and it would have produced a catalog whose titles disagreed with what the player sees on their own screen — a bug with no error message anywhere.

Confirmations, where YARG agreed with decisions we had reached from source:

- Duplicate keys resolve **last-wins** — `FIRST` never appears in YARG's cache, `SECOND` does. That is now three independent sources: `IniModifierCollection.cs` , SngCli's round-trip, YARG.
- Latin-1, UTF-16LE and UTF-8-BOM `song.ini` files all read correctly on both sides.
- `notes.mid` is a **hard selection** over `notes.chart` , not a preference. Given both, YARG chose the `.mid` , found it unplayable, and rejected the whole song rather than falling back. Hashing the wrong file would be a real divergence, exactly as `chartfile.go` warns.
- Uppercase `[SONG]` , ragged whitespace, CRLF, values containing `=` , free-text years, unknown keys, the `cover` override, clean/explicit stem variants and a 4-way drum kit all matched.

Both divergences are now closed

1. ~~**UltraStar `notes.txt` .~~ CLOSED.** YARG rejected it with "*Name metadata not provided*" despite `song.ini` carrying a name, and the reason turned out to be structural: **UltraStar is the one format whose metadata does not come from `song.ini` .** YARG reads `#TITLE` , `#ARTIST` and `#ALBUM` from the `.txt` itself and refuses the song outright when `#TITLE` is missing. It is also **vocals-only** — no instrumental part is derived from it, and harmony is keyed on `#PARTS:2` .

Re-measured with corrected fixtures: a valid UltraStar chart is accepted and **YARG titles it from the `.txt` , not from `song.ini`** — the corpus case deliberately carries a different name in each, and `song.ini` 's never appears in YARG's cache. Our scanner now does the same, reports lead vocals (plus harmony on `#PARTS:2`), and flags a titleless chart as

`ultrastar_no_title`.

1. ~~**We do not know whether a chart is playable.**~~ **CLOSED** by the chart preparsers. YARG rejected the header-only `.mid` with *"No notes found"*; our preparser now reports zero parts for that same chart, reaching the same conclusion by an independent route. A song with no detectable parts is still catalogued rather than dropped, but it no longer claims instruments it does not have.

The writer's gate, 2026-09-05

The same oracle was then pointed at output we produced. `yarg-song-server pack` repacked 14 corpus folders plus the reference song; YARG scanned the 15 archives and accepted 14, reading every title out of our metadata section.

The single rejection — `No notes found` — was settled by a **control** rather than explained away: SngCli's own archive of the same source folder was placed beside ours and scanned in the same pass, and YARG rejected both with the identical error. The fixture's hand-written chart has no note events. Without that control, "YARG rejected one of ours" would have looked like a writer bug, and chasing it would have been wasted work.

Independently, `SngCli decode` extracted our archive with zero errors and every contained file came back byte-identical to the source folder.

Reproducing this

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus
# point YARG at it by editing SongFolders in
#   %USERPROFILE%\AppData\LocalLow\YARC\YARG
elease\settings.json
# then delete songcache.bin, launch YARG, and read badsongs.txt beside it
```

The corpus is generated, never committed: it is large, and a committed corpus rots into something nobody regenerates.

The gap this leaves, stated honestly

Self-authored input cannot produce unknown unknowns. We can write a `song.ini` with the mistakes we thought of; we cannot write one with the mistakes we did not. Two things partly close that gap:

- SngCli's output already taught us two things synthetic input never would — that duplicate keys resolve last-wins in the reference tool, and that a contained file's extension can lie about its container.
- YARG scanning our output is a real oracle, because it is the actual client with the actual parser.

Neither is a substitute for a large real library. If a legitimately-licensed corpus becomes available, revisit this.

That said, the first run of this method found three real bugs, two of which had been written into the code as confident comments. The oracle is doing work.

Second oracle run — 2026-09-05, against what the SERVER produced

The first run scanned loose folders. This one scanned the archives the running server handed a client, which is the thing Phase 2 actually promises: *an unmodified YARG reads what the server serves.* The two runs answer different questions and both are worth keeping.

Procedure, end to end and all of it measured:

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus     # 22 cases at the time; 23 now
go build -o yss.exe ./cmd/yarg-song-server
.\yss.exe -listen 127.0.0.1:8099 -songs $env:USERPROFILE\yarg-test\corpus -data .\data
# POST /api/v1/have with an empty list -> 22 missing
# GET /song/{hash}.sng for each -> 22 files
# point YARG's SongFolders at the downloaded folder, delete songcache.bin, launch, wait
```

Results:

Step	Outcome
Index the corpus	22 songs, 22 distinct charts, 0 problems, 15 ms
`POST /have` from an empty client	`library_total=22 missing_count=22`
Download every missing song	22 downloaded, 0 ambiguous, 0 failed
Re-scan the downloads with our own scanner	22 songs, 0 failures
YARG scans the 22 served archives	**20 accepted, 2 refused**

The two refusals, and our own independent verdict on the same archive:

Archive	YARG said	Our scanner said
`30334065…` "Mid Beats Chart"	`No notes found`	**nothing — no issue at all**
`4ec68053…` "No Audio"	`No audio accompanying the chart file`	issue `no_audio`, no stems

> **This section originally claimed the standard held, and it did not.** The row above used to read "*`parts_derived: true`, no part carries any difficulty*" — which is an observation somebody made by reading the JSON, not an issue the scanner raised. The scanner emitted **no issue whatsoever** for `30334065…`. Reading a field by eye and recording it as though the software had flagged it turned a miss into a pass, and the conclusion drawn from it — "the standard holds" — was false for one of the two songs it was about. Corrected on the third run below, where the gap was found and closed. The lesson is narrower than "check your work": **a green must come from the thing being tested, not from the tester noticing something.**

What this run turned up: packing gives an untitled UltraStar chart a title

Corpus case `21-ultrastar-no-title` has a `notes.txt` with no `#TITLE` and a `song.ini` that does carry a name. As a **loose folder** YARG refuses it — UltraStar is the one format that takes its title from the chart rather than from `song.ini`, and a missing `#TITLE` is fatal ("Name metadata not provided"). Our scanner flags it `ultrastar_no_title`.

As a `.sng` **produced by this server**, YARG accepted it. That is not a packing bug: in a `.sng` the metadata lives in the archive's header section, and `PackDir` fills that section from `song.ini`, so the song genuinely has a title — scanning the archive shows its name resolved to "Has A Name In song.ini".

Correction, measured 2026-09-05 on the third run: this paragraph used to end "*Our scanner reports no issue for the archive, which agrees with the client in both forms.*" It does not. Serving the packed archives and reading `/api/v1/songs` returns `ultrastar_no_title` on the archive as well, because the `notes.txt` inside it still has no `#TITLE`. That is a caveat on a song YARG plays, not a claim that YARG refuses it, and the issue text now says so in both directions. Worth keeping rather than suppressing: the chart really is missing a tag, and a player repacking it elsewhere would meet the folder behaviour again.

It is worth writing down anyway, because it means one sentence people will reasonably assume is not quite true: **the server does not always serve exactly what the folder was.** For this one format, repacking makes a song playable that was not. Nothing to fix; something to know before the Phase 2b sync client starts writing archives into a player's songs folder and a song appears that the player could not previously play.

Third oracle run — 2026-09-05, against a folder `yarg-sync` filled

The first run scanned loose folders. The second scanned archives downloaded by hand from the running server. This one is the Phase 2b exit criterion itself: **the real sync client filled an ordinary songs folder, and an unmodified YARG was pointed at it.** Nothing between the server and the game was done by hand.

```
.\bin\yarg-song-server.exe -songs $env:USERPROFILE\yarg-test\corpus -data .\data -listen 127.0.0.1:8099
.\bin\yarg-sync.exe -server http://127.0.0.1:8099 -songs $env:USERPROFILE\yarg-test\phase2b\synced
# point YARG's SongFolders at that folder, delete songcache.bin, launch, wait ~45s,
# then read badsongs.txt beside it
```

Step	Outcome
Index the corpus	22 songs, 22 distinct charts, 0 problems
`yarg-sync` from an empty folder	22 downloaded, 0 failed, 229,515 bytes in 505 ms
Every archive verified by re-derived identity	22 / 22
YARG scans the synced folder	**20 accepted, 2 refused**

The refusals were the same two, for the same reasons, at the paths the client wrote:

```
C:\Users\ENG2\yarg-test\phase2b\synced\30334065…sng   No notes found
C:\Users\ENG2\yarg-test\phase2b\synced\4ec68053…sng   No audio accompanying the chart file
```

So the Phase 2b claim is now measured rather than asserted: an unmodified YARG plays from a folder this client filled, and it does so with no change to the game.

The fifth thing the oracle found, and the fourth defect

`13-mid-beats-chart` is a `notes.mid` carrying a beat track and no note tracks. YARG refuses it — `No notes found`. **Our scanner reported it clean.** Not "flagged with a different code": no issue at all. That breaks the standard this project exists to hold, and it had been recorded as a pass on the previous run (see the correction above).

Chasing it turned up a second, deeper defect. Two corpus cases are byte-identical apart from one line:

	`20-ultrastar`	`21-ultrastar-no-title`
`#TITLE` line	present	**absent**
every other byte	identical	identical
difficulties we derived	8	**0**

`PreparseUltraStar` returned early without deriving any part when `#TITLE` was missing, on the belief — written into the code and into a test — that YARG refuses such a chart outright, so its contents did not matter. **The second oracle run had already disproved that belief** (packed, the song plays) without anyone noticing that a function elsewhere still depended on it. So the server reported a song with no playable parts, for a song that plays.

That also made the obvious fix for the first defect wrong. A blanket "zero difficulties means no notes" rule would have flagged `21-ultrastar-no-title`, which YARG accepts — a false positive created by building on a known-bad number.

Both are fixed:

- `PreparseUltraStar` derives notes regardless of the title. What a chart *contains* does not depend on whether it is titled. The missing tag is recorded as a note, and the title question stays where it belongs, in the metadata layer.
- A new issue, `no_notes`, is raised when a chart parses cleanly and no instrument carries any difficulty. It is guarded on `PartsDerived`, because zero difficulties on a chart we *failed* to parse means "not determined" — the catalog already documents that distinction on the field itself, and claiming "no notes" there would report a conclusion never reached.
- `ultrastar_no_title`'s text asserted that YARG "refuses the song". It now states both measured behaviours: refused as a loose folder, played once packed.

Each change was red-proofed before its green was believed. Removing the `no_notes` check failed `TestNoNotesIsFlaggedBecauseYARGRejectsIt` and nothing else; dropping the `PartsDerived` guard failed `TestNoNotesIsNotClaimedWhenPartsWereNeverDerived` and nothing else; restoring the UltraStar early return failed both UltraStar tests and nothing else.

After the fix, the corpus reads:

Corpus case	Difficulties	Issues	YARG
`13-mid-beats-chart`	0	`no_notes`	refused
`19-no-audio`	8	`no_audio`	refused

Corpus case	Difficulties	Issues	YARG
`21-ultrastar-no-title`	**8** (was 0)	`ultrastar_no_title`	accepted
the other 19	8 or 16	none, or a metadata caveat	accepted

The two songs YARG refuses are exactly the two this scanner flags as refusals. Song identity was re-checked and is unchanged by all of this — `SHA1(chart bytes)` does not move because our derivation improved.

Fourth oracle run — 2026-09-06, TWO machines, and the defect only two machines could find

The third run closed the Phase 2b claim for one client. This one is the exit criterion in full: **two different computers, each running an unmodified YARG, each pointed at a folder `yarg-sync` filled from the same server.** It is the first run in this project where the question was not "does the client accept what we serve" but "do two clients receive the *same thing*".

They did not.

	ENG-1	r7-desktop
Archives received	22	22
Total bytes	229,515	229,515
Songs failed	0	0
Archives with matching SHA-256	**6 of 22**	

Same count, same byte total, same server — and 16 of the 22 files were different. The count and the total are exactly the numbers a summary line reports, and both were green. Only hashing every file showed it.

The cause, and why every earlier run missed it

`sng.Write` generated its 16-byte header mask with `crypto/rand` on every call, so `PackDir` was not a function of its input. 16 was not a coincidence: the server was running with a deliberately tight `pack_cache_max`, 16 of the 22 songs had been evicted and re-packed between the two syncs, and a re-pack meant a new mask.

Three things had hidden it:

- **A cache hit is not a re-pack.** `TestSongFileIsAReadableSNGWithTheSameIdentity` asked for the same song twice and compared the bytes — but the second request was served from the cache, and a hit re-reads a file rather than packing anything. It passes whatever the packer does. It still passes today with the defect deliberately reintroduced; the new `TestTheSameSongIsTheSameBytesAfterTheCacheIsEmptied` fails, which is how we know the difference is real and not a matter of taste.
- **A test asserted the defect.** `TestWriteUsesAFreshMask` required two writes of the same content to differ, on the belief that a repeated mask weakened the obfuscation. It does not: the mask is stored in plaintext in the header of every `.sng`, so it protects nothing at all. The test locked the bug in and made removing it look like a regression.

- **The claim was reasoned, not measured.** Six documents said an evicted archive is re-packed *byte-identically*. That was inferred from "the package hash comes from the content", which is a fact about the hash and says nothing about the archive. Nobody had compared two archives.

The fix and the measurement

The mask is now derived: `key = SHA-256("yarg-song-server/sng-mask/v1\0" + package_hash)[:16]`, and `sng.Write` takes the key as a parameter instead of drawing it itself. Nothing is given away — the mask is in the header regardless — and packing becomes a pure function of the folder.

Measurement	Result
Pack the same folder twice in-process	identical
Sync, wipe the server's entire pack cache, sync again	**22 / 22 identical**
ENG-1 vs r7, same server, independent syncs	**22 / 22 identical** (was 6 / 22)
Unmodified YARG on ENG-1	**20 accepted, 2 refused**
Unmodified YARG on r7-desktop	**20 accepted, 2 refused**

Both machines refused the same two songs for the same two reasons — `No notes found` on `30334065…` and `No audio accompanying the chart file` on `4ec68053…` — which are the two our own scanner flags. r7 had never run YARG before; its install was copied from ENG-1 and its `settings.json` written from ours with only `SongFolders` changed.

So the Phase 2b exit criterion is closed, and closing it is what found the defect. A single client cannot notice that it received one of several possible encodings.

(Footnote worth keeping: the two `songcache.bin` files are 6,619 and 6,606 bytes. Same 22 songs, same verdicts, different sizes — they embed absolute paths. That is a small live demonstration of why generating `songcache.bin` is a permanent non-goal.)

The container leg, measured the same day

vault2 was re-pulled onto `c623dae` with its pack cache wiped, and both clients synced from the container as well. Five independent syncs, hashed file by file:

Client	Server	Result
ENG-1	ENG-1, from the working tree (windows/amd64)	reference
ENG-1	same, after wiping the whole pack cache	identical
r7-desktop	ENG-1's server	identical
ENG-1	the vault2 container (linux/amd64)	**identical**
r7-desktop	the vault2 container (linux/amd64)	**identical**

110 archives, two clients, two servers built for different operating systems by different toolchains, one deliberate cache wipe — one set of bytes. **The derivation turns out not to be platform-dependent**, which it had to be for any of this to mean anything and which nothing had actually shown. Unmodified YARG on the container's output: 20 accepted, 2 refused, the same two.

Details of the redeploy are in `docs/DEPLOY-VAULT2.md`.

Fifth oracle run — 2026-09-06, a song served from inside a `.zip`

`.zip` / `.7z` ingest landed, so the corpus gained a **23rd case,** `23-zipped` — an ordinary song delivered as `23-zipped.zip` containing `23-zipped/`, which is how songs are actually distributed. The song inside is deliberately unremarkable: the case under test is the container, and a case probing two things at once cannot say which one failed.

Step	Outcome
Index the corpus	23 songs, 23 distinct charts, 0 problems, 14 ms
The zipped song in the catalog	`name="Zipped"`, `source_path=23-zipped.zip`
`yarg-sync` from an empty folder	23 downloaded, 0 failed, 238,215 bytes
Unmodified YARG on the synced folder	**21 accepted, 2 refused**
Issues our own scanner raises, refusal-class	exactly 2 — the same two
Issues on the zipped song	none

The two refusals are the same two as every run since the corpus existed — `No notes found` on `30334065…` and `No audio accompanying the chart file` on `4ec68053…`. **The song that arrived inside an archive was accepted, and by the time it reached the player's folder it was indistinguishable from any other**: the client writes `<chart_hash>.sng` regardless of what the server ingested it from.

`.7z` is covered by `TestA7zSongPacksToTheSameBytesAsTheLooseFolder` rather than by the corpus, because **Go cannot write a 7z** — the library is read-only. The fixture at `internal/scan/testdata/song.7z` is committed, and was built with `py7zr` from exactly the bytes the loose-folder case uses. Regenerate it from anything else and the test fails for a reason that has nothing to do with 7z.

The strongest assertion of the set is `TestAllThreeContainersProduceTheSameArchive`: a loose folder, a zip of it and a 7z of it all pack to one archive. That is what lets an operator restructure a library without invalidating every client's copy of every song — the same property the random-mask defect broke, arrived at from a different direction.

What is still not measured

One real YARG version, on two Windows machines, against a corpus we wrote ourselves. The corpus is adversarial by design, but it is still ours: it cannot contain the case nobody thought of. A larger body of real community charts would test different things, and the licensing constraints on where those may come from are in the section above.

Both machines run the same Windows build of the same YARG version, so nothing here says anything about Linux, macOS, or a different client release.

The ARM leg — 2026-09-06, and the determinism result crosses an architecture

The arm64 half of "portable to Raspberry Pi" had never been executed; it rested on ELF machine type and image config, which are checks on bytes. It has now run, on a **Raspberry Pi 4 Model B Rev 1.5**, `aarch64`, Debian 13 trixie.

What ran	Evidence
The cross-compiled `linux/arm64` binary	the Pi's own `file`: `ELF 64-bit LSB executable, ARM aarch64, statically linked`; 22 songs indexed in 24 ms
The published multi-arch container image	`arch=arm64`, image id `3c5fdd28…`, `version=1ae2219`, 22 songs in 25 ms, docker.io 26.1.5

Then both were synced from, and compared against the reference:

Client	Server	Result
ENG-1	ENG-1, working tree (windows/amd64)	reference
ENG-1	same, after a full cache wipe	identical
r7-desktop	ENG-1's server	identical
ENG-1	vault2 container (linux/amd64)	identical
r7-desktop	vault2 container (linux/amd64)	identical
ENG-1	**Pi, bare binary (linux/arm64)**	**identical**
ENG-1	**Pi, container (linux/arm64)**	**identical**

Seven independent syncs, two operating systems, two CPU architectures, 154 archives, one set of bytes. The mask is derived with SHA-256 over a domain string and the package hash, and nothing had shown that this is stable across architectures — it was assumed, which is the same species of assumption that produced the original defect. It now holds by measurement.

Three caveats, stated rather than buried:

- The board was a **Pi 4, not the 3B+** this project had been waiting on. That 3B+ is dead: steady red PWR, no ACT activity at all, across three independently written cards — one from Imager, one written sector-by-sector and verified by reading it back, and one written by a freshly installed Imager. Zero ACT means the SoC never reads the card, so no card could have fixed it.
- The Pi was **one-shot and wiped afterwards**. This is "arm64 ran on this date", not a maintained deployment, and the claim should be re-measured rather than inherited.
- The image reached the Pi by `docker save` on vault2 and `docker load` on the Pi rather than a direct registry pull, to avoid putting a registry credential on a throwaway host. The image id is identical either way, so what ran is the published image; only its transport differed. The multi-arch manifest itself was separately confirmed to carry both `amd64` and `arm64`.

The macOS leg — 2026-09-08, and it runs on every commit rather than once

macOS was the last platform this project claimed and had never executed. Two darwin binaries came out of every pipeline for weeks; everything said about them was inference from a successful cross-compile.

There is no Mac here, and renting one by the hour proves the build it was rented for and nothing after it. `.github/workflows/macos-verify.yml` runs on GitHub's free macOS runners instead — the one workflow that lives on GitHub, still committed to origin and carried downstream by the mirror, because GitHub is the only place this project can reach a Mac.

First result, run #1 on `cb2a2ef`:

What ran	Evidence
A native Apple-silicon build	`go version go1.27.1 darwin/arm64`; the runner's own `file`: `Mach-O 64-bit executable arm64`; `Darwin Kernel 25.6.0 … RELEASE_ARM64_VMAPPLE arm64`
The server, against the generated corpus	`songs=23 problems=0`
`yarg-sync`, over HTTP	`downloaded 23 song(s) … 0 failed, 238215 bytes`
A rescan of what it downloaded	`identical chart-hash sets: 23`
Agreement with Linux	the `agree` job compared both hash manifests and found no difference

238,215 bytes is the same total the Windows, Linux and Unity-batchmode runs produce for this corpus, so the macOS leg lands on the number the rest of the project already agreed on rather than merely being self-consistent.

Two things this deliberately does NOT claim:

- **It is not the release artifact.** The job builds natively on the runner; the shipped darwin binaries are cross-compiled on Linux. This proves the code runs on macOS, not that the exact file in `release/` does. Closing that would mean checksumming a cross-compiled binary against a natively built one, and Go does not promise those match.
- **Nothing here has been run by a person on a Mac.** No Finder, no Gatekeeper prompt, no double-click. A CI runner executes the binary from a shell with the quarantine attribute never applied, which is not the first-run experience a downloader gets.

The assertions were red-proofed on Linux before the workflow was trusted — a corrupted `.sng`, a missing `.sng`, an unserved chart, two disagreeing manifests and a missing leg each produce the failure they are meant to. That local run also killed a wrong assertion in the first draft, which had the ETag equal to the sha256 of the downloaded archive. It is not: it is the package hash.

Hostile archives — 2026-09-06, probed rather than reasoned about

The oracle runs above validate songs that are *supposed* to work. Archive ingest also has to survive files that are not: an operator's library is downloaded from the internet, so the reader is attacker-facing.

None of these behaviours were asserted from reading the standard library. A throwaway `probe_test.go` was written that simply *printed* what happened for each hostile shape, the output was read, and only then were tests written against what was observed. The probe was deleted; what it found is pinned in `internal/scan/hostile_test.go`.

Shape fed in	What actually happened	Now pinned as
`../../../../etc/passwd`, `Song/../../escape.txt`, `/etc/shadow` alongside a real song	all three dropped by `fs.ValidPath`; produced `.sng` held only `song.ogg`, `notes.mid`, `album.png`	traversal never reaches the served archive
truncated / non-zip bytes with a `.zip` name	reported cleanly, walk continued, the good song beside it still found	one bad file does not abandon a scan
zip of unrelated files (photos)	`ErrNoChart`, silently skipped	quiet, so real problems are not buried
empty zip	`ErrNoChart`	not a song, not a problem
zip containing a `.sng`	`ErrNoChart`	not descended into
zip with backslash separators (`Song\song.ini`)	**`rootNames` empty, `rootDirs` = `[Song]`, `ErrNoChart` → silently ignored**	**`ErrUnreadableArchive`, reported**

The last row is a real defect, and only the probe found it — a legitimate song disappearing from a library with no message, the same class of failure as silently skipping a `_rb3con`. The reasoning behind the fix, and why the first attempt at it was wrong, is in ADR-003.

Red-proofed, not just green

Each of these tests was confirmed to fail when the behaviour it guards is removed — a test that has never been red proves nothing:

Mutation applied to the source	Test that failed
`looksLikeASong` → always `false`	`TestAnArchiveThatVisiblyHoldsASongIsNeverSilentlyIgnored`
`looksLikeASong` → always `true`	`TestAnArchiveOfNonSongFilesIsStillIgnoredQuietly`, `TestAnEmptyArchiveIsNotASong`
`WalkLibrary` swallowing every container error, not just `ErrNoChart`	`TestACorruptArchiveIsReportedAndTheWalkContinues`, `TestAnArchiveThatVisiblyHoldsASongIsNeverSilentlyIgnored`

Both directions of `looksLikeASong` matter, which is why both were mutated: too strict and songs vanish, too loose and every holiday-photos zip becomes a problem entry.

`TestTraversalEntriesNeverReachTheServedArchive` is deliberately **not** red-proofed, and it is honest to say why: the behaviour it pins belongs to Go's `archive/zip`, not to this codebase, so there is no line here to mutate. Its value is not that it catches our regressions — it is that it fails loudly if a toolchain upgrade ever changes a filter we now depend on for a security property.

Sixth oracle-adjacent run — 2026-09-06, the fix observed on the deployment

Two properties were measured on the vault2 container after it was upgraded to `077f36e`, because neither is reachable from a unit test.

Determinism now holds across a code change. Every earlier determinism result compared machines, operating systems or architectures at a *fixed* commit. Here the pack cache was hashed under `423902b`, wiped, and re-packed by the `077f36e` binary: **23 archives, byte-for-byte identical**. A `yarg-sync` from ENG-1 then delivered 23 files totalling 238,215 bytes, and the set of client-side SHA-256s matched the set of server-side pack SHA-256s exactly. That is the property an `ETag` and a `Range` resume actually rely on in practice, since a server is upgraded far more often than it changes CPU.

The silently-ignored song was reproduced on the deployment and observed being reported. A probe archive holding `song.ini`, `notes.mid` and `song.ogg` under `Backslash Song\` was placed in the host library. The server indexed 23 songs with `problems=1`, naming the file and saying what to do about it, in the log and in `/api/v1/library` both. Under the previous image the same file gave `problems=0` and no mention anywhere. The probe was removed and the library restored to 23 clean cases.

Building that probe turned out to demonstrate the defect twice over. Python's `zipfile` rewrites `os.sep` to `/` inside `ZipInfo.__init__`, so the obvious construction silently produces a forward slash and tests nothing; and `namelist()` normalises backslashes on read, so a correctly-built probe still *looks* wrong. Counting `0x5C` bytes in the file is the only honest check. **Both the writer and the reader hide the thing under test** — the same shape as the original defect, where an `fs.FS` view of an archive is not the archive.

Sixth oracle run — 2026-09-07, the first against REAL community charts

Every earlier run scanned songs this project wrote. Jay supplied three community song packs — **128 songs, 1.16 GB** — and they were run through both YARG and our scanner. This closes the gap `docs/SCALE.md` and the section above both name: *self-authored input cannot produce unknown unknowns.*

It also makes the oracle a script rather than a hand procedure. `scripts/oracle.ps1` repoints YARG's `SongFolders`, wipes the cache, launches, waits, compares, and **restores** `settings.json` **in a** `finally` **block** so a crash still puts the operator's YARG back.

The result, and why it is a WEAK positive

Songs in library	128
YARG refused	**0**
Our scanner flagged	**0**
YARG refused and we passed	**0**
We flagged and YARG accepted	0

The standard held — but it held *trivially*. **Both sides said everything was fine, so there were no disagreements available to find.** A curated pack that people actually play is close to the least informative sample possible for this test: it contains no broken songs, which is precisely what the comparison exists to catch. Reporting this as "the oracle passed at scale" would be the same error as the second oracle run, which recorded a miss as a pass.

What it *does* establish, and these are worth having:

- **Zero false positives on 128 real charts.** Nothing had ever tested that. A scanner that cried wolf on ordinary community songs would be useless in exactly the situation it is for, and this is the first evidence it does not.
- **The harness works end to end and refuses to lie.** It waits on the *song cache* rather than on `badsongs.txt`, because a library YARG is happy with produces no `badsongs.txt` at all — waiting on that file would hang on the very outcome we most want to report. It requires the cache's mtime to be newer than the launch, so last run's file cannot be read as this run's verdict, which is a mistake this project has already made once.
- `ErrTooManySongs` **is right about real files.** Pointed at the three pack `.zip`s as downloaded, the scanner refused all three with "*archive holds more than one song folder; unpack it and add the songs individually*". That behaviour existed only as a synthetic test until now.

A second, independent oracle: upstream's own scanner

`YARG.Core.UnitTests` has two integration tests, `FullScan` and `QuickScan`, that were being **skipped** — they call `CacheHandler.RunScan` over directories named by the `YARG_TEST_SONG_DIRS` environment variable, and skip when it is unset. Pointed at the same 128 songs, both pass and YARG.Core writes **no** `badsongs.txt`.

That is worth more than one more green tick: it exercises upstream's real scanner in-process, without launching the game, so it can run in seconds and on any machine with the .NET SDK. Note what it does and does not assert — upstream's own comment says "*the only fail condition would be an unhandled exception*" — so it is a crash test over real input, not a verdict test.

What is STILL not covered

- **All 128 songs are** `.mid`**.** Real-world `.chart` coverage remains zero, and `.chart` is the format with the early-return bug this project already found once.
- **No broken songs.** The sample most likely to find a defect is old, odd and badly made — Frets on Fire era, odd encodings, missing stems, non-Latin metadata. A deliberately ugly hundred would be worth more than these curated packs, and than a thousand more like them.
- **The corpus is not redistributable and is not committed.** These are commercial recordings charted by the community; they live on one machine and only statistics reach this repo. That is the same line drawn at the top of this document, and it is why `cmd/mkscale` exists.

Seventh oracle run — 2026-09-07, songs broken on purpose, and the standard finally fails

The sixth run held trivially: 128 healthy songs, nothing refused, nothing flagged. A comparison where both sides say nothing cannot tell a scanner that agrees with YARG from one that is asleep.

So `cmd/mkbroken` takes songs with **real structure** and damages them in named ways — each case carrying a **predicted verdict**. That turns the oracle from a two-column table into three:

```
prediction | our scanner | YARG
```

and each disagreement means something different. YARG refusing what we passed is the standard violated. YARG *accepting* what we predicted it would refuse means our model of YARG is wrong, which a two-column comparison cannot see at all.

16 cases were generated from the 128-song library. **Neither the input nor the output may be committed**: damage a copyrighted recording and it is still a copyrighted recording. The tool ships; what it makes does not.

The result

Songs indexed	14
YARG refused	6
We flagged	5
YARG refused, we PASSED	**2**
We flagged, YARG accepted	1

The two we missed:

Case	YARG's reason
`03-chart-truncated`	Corruption of either the ini file or chart/mid file
`04-chart-is-not-a-chart`	Corruption of either the ini file or chart/mid file

`03` is a MIDI cut to a third of its length. `04` is a plain text file named `notes.mid`. **Our scanner indexed both as perfectly good songs and reported no issue at all**, and served them to a client that then refuses them.

That is the standard this project holds, violated, and found by the corpus on its first run.

The diagnosis in the paragraph above was wrong, and being wrong about it mattered

It said the scanner "never parses the chart, it only hashes it" and so "cannot tell a chart from a shopping list". Measured on 2026-09-07 with a probe that printed every issue on eleven damaged shapes, which is what should have been done before writing a cause down at all:

Shape	What the scanner actually did, before any fix
plain text named `notes.mid`	**parsed it, failed, and recorded a note** — then indexed it with no issue
empty `notes.mid`	same
`notes.chart` holding a shopping list	**already flagged**, `no_notes`

Shape	What the scanner actually did, before any fix
nine-track `.mid` cut to a third	parsed, **found real instruments**, no note, no issue

So the scanner *does* preparse every chart — `applyPreparse` has called `PreparseMIDI` / `PreparseChart` since Phase 1. Two things were true instead, and they need different fixes:

1. **For a chart it cannot read at all, it detected the problem and then said nothing an operator or a client would see.** The failure became a cosmetic `parts_note`, not an issue. *Detecting something and reporting nothing is indistinguishable from not detecting it.*
2. **For a truncated chart, nothing detected anything** — and this is the case the proposed `MThd`-magic check would have missed, exactly as the previous entry warned. A real chart cut in half keeps a valid header and several **complete** `MTrk` chunks, so it preparses cleanly and reports genuine instruments. It looks perfect.

The `.chart` half of the proposed check turned out to be unnecessary: a `.chart` that is not a chart yields no sections, so `no_notes` already catches it. **Half of the fix that was about to be written was already there, and the half that mattered was not the half that had been designed.**

The fix, and what it does not cover

- `chart_unreadable` — the preparse failed. Was a note; is now an issue.
- `chart_truncated` — the file ends inside a chunk it declared. This is the only evidence of truncation that survives in a real chart: the file contradicting its own chunk lengths.

It does **not** catch a cut landing exactly on a chunk boundary. Such a file is byte-for-byte a valid chart with fewer tracks, and nothing short of the original could tell them apart. That is stated as a test, `TestACutOnAChunkBoundaryIsNotDetected`, rather than as a comment nobody re-reads.

False positives: measured, not hoped

A check that fires on healthy charts is worse than no check, because it would flag most of a real library. Run over all **270 songs** in the real corpus at `C:\dev\_incoming\YARG`:

```
songs=270 flagged_by_new_checks=2
  FLAGGED broken/03-chart-truncated        chart_truncated
  FLAGGED broken/04-chart-is-not-a-chart   chart_unreadable
```

The only two flagged are the two that were damaged on purpose. Zero healthy songs tripped either check.

Run 8 — the standard held

Re-run of the same 16 cases after the fix:

Songs indexed	14
YARG refused	6
We flagged	7

YARG refused, we PASSED	**0**
We flagged, YARG accepted	1

> `STANDARD HELD: every song YARG refused was one we independently flagged.`

The one in the other column is unchanged and is the allowed direction: `07-no-song-ini` is flagged by us and loaded by YARG.

The one in the other column is fine: `07-no-song-ini` is flagged by us and loaded by YARG. A scanner is allowed to be stricter than a refusal.

The harness lied first, and the fix changed the answer

The first run of this reported **3** rejections and 3 misses. The corrected run reports **6** and 2. The difference was not YARG.

`badsongs.txt` reports some rejections against the offending **file** rather than the song folder, so keying the comparison on `Split-Path -Leaf` produced a key of `notes.mid` — which matched no case, collapsed four separate rejections onto one key, and made the songs they belonged to look accepted. **The table still added up. It was still wrong.**

Both sides now reduce any path to the song folder relative to the library root, and the script reports **unattributable rejections** explicitly and exits 2 — inconclusive rather than passing — whenever a refusal cannot be matched to a song. A comparison that cannot name what it compared should say so, not average it away.

That is the fourth instrument failure in this stretch of work, after the length-comparison probe, the socket-exhausted load harness, and the Windows rename-while-open reasoning. The pattern is consistent enough to state as a rule: **when a measurement surprises you, suspect the instrument before the subject** — and when the instrument is fixed, re-read the result, because it may have been wrong in both directions at once.

Run 9 was not run, and that is the stronger result

2026-09-08. The scanner changed after run 8 — `667e507` extracted `scan.ScanFile` out of `WalkLibrary` so the upload-check endpoint and a library walk could not disagree about what a file is. A dispatch refactor is exactly the shape of change the oracle exists to catch, so run 9 was the obvious next move.

It could not be driven from this session, for two honest reasons rather than one:

- **A detached process on ENG-1 cannot read YARG's `release` profile.** Measured: a process created through `Win32_Process.Create` runs as the same user, at medium integrity, and sees `…\LocalLow\YARC\YARG` containing only `dev`. The bridge session sees `dev` AND `release`. The oracle needs `release`, because that is where the real v0.15.0 game keeps `settings.json`, `songcache.bin` and `badsongs.txt`.
- **`oracle.ps1` needs up to 180 s and a bridge call dies at about 45.** It repoints `settings.json` and restores it in a `finally`, so a call killed mid-run would leave the operator's YARG pointed at the test library. A scheduled task registered with an interactive principal stayed `Queued` and never started.

So the question was answered a different way, and the answer turns out to be better than a run.

Both sides of the comparison are provably unchanged

The oracle compares **what YARG rejects** against **what our scanner flags**. Run 9 would have re-derived both. Instead each was shown not to have moved:

Our side — measured by diffing the scanner against itself. `662907b` (the commit before the refactor) and `9ec5dfb` (current) were both built and run over the same two libraries:

Library	Before	After	Difference
128 real community songs	128 songs, 0 failures, 22,370 lines of JSON	identical	**none**
the 16-case `mkbroken` corpus	14 songs, 0 failures	identical	**none**

Byte-identical on stdout and stderr both times, once `date_added` is normalised out — it is `time.Now()` and changes every run, which is why a naive file comparison shows a seven-byte difference and means nothing. The JSON carries the `issues` array, so *what we flag* is included in that identity, not just *what we index*.

YARG's side — unchanged by construction. Same binary (v0.15.0, the release build), same corpus: all 69 files under `broken/` were last written 2026-09-07 00:19, before run 8, and the manifest still lists its 16 cases.

Same inputs, same programs, so the same verdicts. **Run 8's result stands for `9ec5dfb`**: 14 indexed, 6 refused by YARG, 7 flagged by us, **0 refused-but-passed**, and the one allowed overshoot (`07-no-song-ini`).

What this does NOT establish

Worth stating plainly, because "equivalent to a run that passed" is not the same claim as "ran and passed":

- It says the refactor changed nothing **on these two libraries**. A dispatch difference that only shows on a shape neither library contains would survive this check — a `.7z`, a console package, an archive with backslash separators. The unit tests cover those; the oracle would not have exercised them either.
- It says nothing about YARG's behaviour on anything new. Nothing new was added to the corpus.
- **The next real change to the scanner needs a real run 9**, on a machine where somebody can watch it. This substitutes for one run, not for the oracle.

A library with a duplicate package, for testing 300 Multiple Choices

The 23-case corpus and the 270-song real library both have `duplicate_packages: 0`, so nothing in either exercises the one response the client cannot handle by ignoring: **300 Multiple Choices**, returned when several packages share a chart hash. It has to be built on purpose.

Two folders with **byte-identical** `notes.chart` and different audio give the same chart hash and different package hashes, which is the whole condition:

```
$root = 'C:\dev\_incoming\dupe-test'
$chart = "[Song]`n{`n  Resolution = 192`n  Name = `"Dupe`"`n}`n[ExpertSingle]`n{`n  768 = N 0 0`n}`n"
foreach ($n in @('pack-a','pack-b')) {
    $d = Join-Path $root $n
    New-Item -ItemType Directory -Path $d -Force | Out-Null
    [IO.File]::WriteAllText((Join-Path $d 'notes.chart'), $chart)   # identical in both
    Set-Content (Join-Path $d 'song.ini') "[Song]`nname = Dupe Song`ncharter = $n`n"
    $bytes = if ($n -eq 'pack-a') { 4096 } else { 8192 }            # different audio
    [IO.File]::WriteAllBytes((Join-Path $d 'song.ogg'), (New-Object byte[] $bytes))
}
```

`/api/v1/library` then reports `songs: 3, distinct_charts: 2`, and `GET /song/<shared hash>.sng` answers **300**.

The audio sizes are the point. They are what lets a test check *which* package arrived rather than only that one did — a pass that proves "no failure" would be satisfied by picking either. Measured 2026-09-07 against the fork's mirror:

package hashes	`66727430…` (pack-b), `7b87f002…` (pack-a)
smallest, so the one the rule selects	pack-b
what the client downloaded	`charter: pack-b`, `song.ogg: 8192`

Read back with the server's own `internal/sng` package rather than a third-party tool — SngCli threw on the archive, and an unexplained failure in the instrument is not evidence about the subject either way.

Neither the corpus nor the server needs to be the live one. This runs against a locally-built `yarg-song-server` on `127.0.0.1`, so it costs nothing and touches no deployment. The generated audio is zero-filled, so unlike the real corpus this one is safe to commit if it ever needs to be.

RETRACTED: "the oracle is a different YARG from the one the fork is built on"

Two sections used to sit here, added 2026-09-07, claiming that current YARG `dev` refuses a song whose `song.ini` omits `song_length` while v0.15.0 accepts it — and drawing from that the larger conclusion that this project's standard had been validated against a build the fork does not use. **Both are withdrawn.** They were a false comparison, and they are replaced rather than quietly deleted because the way they went wrong is the useful part.

The comparison

	how it was run	audio backend
v0.15.0	the real game, via `scripts/oracle.ps1`	**up**
`dev`	editor batchmode, via `ScanFolderProbe`	**dead**

The harnesses differed in the one way that decides this exact case. `SongEntry.IniBase.cs:286`:

```
if (entry._metadata.SongLength <= 0)
{
    using var mixer = entry.LoadAudio(0, 0, false);
    ...
}
```

No `song_length` means the scanner measures the length **from the audio**, which needs `GlobalAudioHandler` — initialised by `GlobalVariables.cs:96` in the running game and never in a static editor method. `LoadAudio` throws, `ScanSngFile` catches it, and reports "*Corruption of either the ini file or chart/mid file*": a message that names neither the real cause nor the real file, and is entirely convincing. **All 14 extra refusals were the harness.**

Calling `GlobalAudioHandler.Initialize<BassAudioManager>()` does not fix it, and does not complain either — decoding a real 3 MB ogg immediately afterwards still throws `NullReferenceException`. `Initialize` constructs the manager and says nothing about whether it works, so "I initialised audio" was never evidence. The probe now decodes a file and believes the result.

What is true, and what is simply unknown

- **Unknown:** whether `dev` refuses songs lacking `song_length`. Not disproven — never tested, because nothing here has yet scanned with `dev` and a working audio backend.
- **True, and mildly interesting:** a song with no `song_length` whose audio cannot be decoded *is* refused with a message blaming the chart. That misdirection is real. It is a diagnostics nit, not a regression, and the upstream question built on the regression claim was pulled rather than sent to a maintainer with a broken control behind it.
- **Unaffected:** the mirror exoneration below. That comparison had the **same** harness on both sides — `.sng` and loose folder refused identically — so it stands.

The rule that would have caught it, stated for next time

Two runs are only comparable when the *harness* is held constant, and "same scanner, same songs" is not the same thing as "same harness". Both runs called `CacheHandler.RunScan` on the same folders; one of them could decode audio and the other could not, and nothing in either output said so. **`ScanFolderProbe` now exits 2 — inconclusive — rather than printing a verdict it cannot support**, which is the same discipline `scripts/oracle.ps1` already applies to unattributable rejections.

It is worth noticing that this project has now made the instrument-versus-subject error five times: the length-comparison probe, the socket-exhausted load harness, the Windows-only rename-while-open reasoning, the `Split-Path -Leaf` oracle key, and this. The pattern is not carelessness about the code — it is trusting a harness that has never been asked to prove itself.

The control that stopped a wrong conclusion being written down

The mirror's own smoke test failed first, with **5 of 23** mirrored songs refused. The obvious reading — *the mirror produces archives YARG will not take* — was wrong, and it had an equally plausible rival that nothing measured could distinguish: that the same songs would be refused as loose folders too.

`ScanFolderProbe` exists to settle exactly that, and it did: **the loose folders were refused identically, same songs, same message.** `.sng` and loose folder behave the same, which is the property the entire remote-library design rests on, and it survived the scare.

Run 9 is withdrawn with the rest

The two-column table that stood here reported v0.15.0 accepting 20 and `dev` accepting 4 over the same corpus. The `dev` column is the harness artefact above and means nothing; the v0.15.0 column (**20 accepted, 3 refused** — `13-mid-beats-chart`, `19-no-audio`, `21-ultrastar-no-title`) was measured in the real game and stands.

The one number worth keeping from that investigation, because it was measured and it is reassuring: **of 256 real community charts on hand, all 256 declare `song_length`.** Whatever any build does with its absence, no real library is affected.

Tenth oracle run — 2026-11-01, the bare CR, and the regression only the oracle caught

A `song.ini` in the wild ends with a **bare CR** — a Mac-classic line ending — splicing two pairs onto one physical line:

```
song_length = 1062206\rplaylist_track = 4
```

That is the last line of `song.ini` in `29 - Take this Crusty L/04 Infant Annihilator - Behold the Kingdom of the Wretched Undying`, inside `Marathon Hero 2.rar`. 671 bytes, the rest of the file ordinary CRLF, no trailing newline. The file is malformed; the parser was not.

It cost the whole song. Our reader kept the CR in the value, so `sng.validateMeta` refused `sng: value of "song_length" contains a newline`, packing failed, and the song answered **HTTP 500**. YARG, meanwhile, plays it and merely loses `playlist_track`. Ours was strictly worse than the game's: YARG drops a field, we dropped the song.

The corpus gained three cases

`24-barecr-value`, `25-barecr-string` and `26-barecr-before-equals`. Each carries **unique sentinel strings**, because `badsongs.txt` only reports refusals and these songs are not refused — the question was what YARG *read*, and the answer is in its own `songcache.bin`, where metadata is stored as length-prefixed UTF-8 and can simply be searched.

Corpus case	Probes	YARG v0.15.0's cache	Verdict
`24-barecr-value`	the real-world shape, a numeric splice	`BareCR Value` cached, song accepted	accepted
`25-barecr-string`	a CR inside a **string** value	`BareCRKeep` present, `BareCRDrop` **absent**, artist `BareCRCorrectArtist`	**truncated at the CR**
`26-barecr-before-equals`	a CR between a key and its `=`	`BareCRBadAlbum` **present,** `BareCRGoodAlbum` **absent**	**YARG reads the pair**

Plus the real song, twice: as a loose folder and as our own repack of it.

Step	Outcome
Corpus cases	26 generated + the real song loose + the real song packed by us
`badsongs.txt`	**Total Errors: 3** — `13-mid-beats-chart`, `19-no-audio`, `21-ultrastar-no-title`
Bare-CR cases refused	**none** — 24, 25, 26, the loose real song and our `.sng` were all accepted
`songcache.bin`	8,885 bytes
SngCli v0.3.0 decode of our `.sng`	`song_length = 1062206`, no `playlist_track`, chart SHA1 `b67974e3…` unchanged

What case 25 settled, and what it refutes

`YARGTextReader.ExtractText`'s non-chart branch does `if (ch == '\r' && stringEnd == -1) stringEnd = container.Position;` and then keeps scanning to the `'\n'`, so a **string** value stops at the first CR exactly as a number does via `TryExtract` halting on a non-digit. `ExtractModifierName` breaks on any character `<= WHITESPACE_LIMIT` (32), and CR is 13. Then `YARGIniReader.IsStillCurrentSection` calls `GotoNextLine` before looking for another key, which is what makes the spliced remainder unreachable.

This **refutes a reading made from `PeekLine`** — that a string value would keep an interior CR and so a blanket truncation would diverge for `KindString`. `PeekLine` is only used by `TrySection` for the `[song]` header; values never pass through it. No `Kind`-aware handling is needed, and `internal/songini/keys.go` was not touched.

Case 26 caught a regression in the fix itself

The first version of the fix cut the whole **line** at the first CR. That is wrong, and nothing in our own test suite could have said so — every test passed.

A CR on the **key** side of the `=` is only whitespace to YARG: `ExtractModifierName` stops on it, `SkipWhitespaceAndEquals` skips it, and the pair is read normally. `26-barecr-before-equals` put `BareCRBadAlbum` in YARG's cache and left the preceding `album = BareCRGoodAlbum` out of it entirely — so cutting the line **dropped a pair YARG reads**, which the previous `strings.TrimSpace` on the key had quietly handled correctly all along.

The fix now cuts the **value**, never the line. This is the clearest case yet of the rule this repo already holds: *a green from our own tests is not a measurement of someone else's code.*

What the fix does, and what it costs

- A value is cut at its first CR or LF, matching `ExtractText`.
- A key carrying `=`, `;`, CR or LF — the set `sng.validateMeta` reserves — is dropped at the reader, where YARG also drops it, since no entry in `SONG_INI_OUTLINES` contains any of them. **The writer's guard is unchanged**; it simply can no longer fire on `song.ini` input. Two further shapes 500'd and nobody had noticed: `na\rme = X`, and any line whose key carries a `;` such as a `; charted by someone = yes` pseudo-comment. (YARG has **no** comment syntax in `song.ini` — it reads `;` as a key and fails the table lookup. A plain `; comment` line with no `=` was never affected.)
- `playlist_track = 4` is **lost**, deliberately, because it is lost in the game. Recovering it would put a number in the browse list that the client does not have, which this project treats as wrong even when it is more sensible.
- It is **not** lost silently. `songini.File.Dropped` carries the text, and the scanner raises `metadata_unreachable` naming it. The song is served; a field is missing; something says why.

How many songs are affected

Library	Scanned	Affected
Live server on vault2	**1,369 / 1,369**	**0**
`Marathon Hero 2.rar`	**152 / 152**	**1** — the song above

Both counts needed their instrument checked first, and one of them needed it three times.

For the live count, `raw_metadata` is `omitempty`, so a zero could equally have meant the field was absent. It is not: 500 of 500 songs in the first page carry it, 14,909 key/value pairs over 129 distinct keys, and **no value in the entire live library contains any control character at all**. The zero is a real zero. Read-only, over `GET /api/v1/songs`.

The archive count took three passes, and the first two were wrong while printing a plausible total:

1. `os.walk` reached **142** of 152. These paths exceed `MAX_PATH`; `os.walk` cannot descend them and does not say so.
2. Taking 7-Zip's own listing as the accounting set reached **143**. 7-Zip's console output mangles non-ASCII folder names, so nine paths could not be matched on disk.
3. Walking from a `\\?\`-prefixed root, and asserting the total against 7-Zip's 152 rather than reporting whatever turned up, reached **152 of 152**. One affected.

A scan that cannot say what it failed to read is not a count. The first two passes would both have supported the sentence "one song in this archive is affected" — correctly, by luck, while being blind to 10 and 9 files respectively.

Still open, and now answerable

`songini.Parse` carries a comment saying its whitespace and multiple-`=` handling is "*the conventional Clone Hero reading*" and **unverified** against `YARGTextReader.ExtractModifierName`. That source has now been read, and it says a key is the maximal run of characters above 32 that are not `=`. So `album track = 5` is, to YARG, key `album` with value `track = 5` — not key `album track` with value `5`, which is what this reader stores.

That is a real divergence and it is **deliberately not fixed here**. It changes how every key containing a space is read, across a whole library, and it is not what made a song unservable. It wants its own change and its own oracle run, with a corpus case of its own. Recorded so the next person starts from a measurement rather than from the comment.

Nine instrument failures in one night — 2026-11-01, and the one rule that covers all of them

Not an oracle run. A measurement post-mortem, recorded here because every one of these was a **correct number read as the answer to a question it was not answering** — and the last of them was correct about a *moment* nobody meant, and the corpus exists to stop exactly that.

The instrument	What it was asked	Why it could not answer
control-character count over `raw_metadata`	"did the bare-CR fix fire?"	the fix *strips* the CR before publishing, so it reads 0 whether the fix works or not
`/api/v1/library` → `archives: []`	"is anything compressed in the indexed set?"	`Archives()` counts `worthUnpacking` = zip|7z|rar only, and is silent about 481 `.yargsong`
`/api/v1/library` grepped for a song name	"is this song present?"	that endpoint returns summary counts and carries no song names at all
`/api/v1/library` → `problems: 0`	"was the song flagged `metadata_unreachable`?"	`Problems` only takes `r.Err != nil`; song issues live in `catalog.Song.Issues` and nothing aggregates one into the other
`os.path.splitext` on `source_path`	"how many songs are loose folders?"	song folders are full of dots — "Mr. Big - To Be With You", "ii (2x bass pedal expert+)" — so ~190 became invented extensions
grep for `playlist_track` on the song payload	"was the field dropped?"	it printed `"playlist_track":16000` — the TOP-LEVEL catalog field carrying the unset default, not the `raw_metadata` key the prediction was about
a local timestamp with `Z` appended	"when did this start?"	`ps -o lstart=` prints the BOX's local time; vault2 is ET, so `07:05:52` was `12:05:52Z`. Writing `07:05:52Z` asserted UTC and made a six-minute job look five hours overdue
`/proc/<pid>/io` → `rchar`	"how many bytes did the index read?"	read at 974s of a 994.49s index, so it was a running total 97.9% of the way through — 4.92 GB short, which turned a ratio of 2.023 into 1.982 and made it contradict the model
`docker manifest inspect` → `denied`	"did CI build this tag?"	a refusal about *authentication* is indistinguishable from "never built"; a control tag that certainly existed returned the same

Most of those were about to be reported as findings, and the `playlist_track` one would have been a false *falsification* of a correct prediction rather than a false confirmation — the same error, pointing the other way. Each would have been a confident wrong answer with a real number attached, and none of them would have looked like a mistake.

The rule: when the question is "did X happen", capture the WHOLE answer and judge the record afterwards — never pre-select the fields you think matter. The witness run for the bare-CR song survived only because it dumped the entire `/api/v1/songs` payload into a variable rather than grepping the four fields someone had decided were the relevant ones. `issues` was not on that list, and `issues` was the witness. The raw dump is what made the fifth miss survivable.

Two corollaries, both of which cost something tonight:

- **An absence is only a reading if the field could have been present.** `issues` missing from a payload that serialises `catalog.Song` directly is a measurement; `issues` missing because nobody asked for it is a gap wearing the same clothes.

- **A unit or a timezone is part of the number.** Two of these were a correct value wearing the wrong label rather than a wrong measurement: `311 MB/s` was bytes per CPU-busy second compared against bytes per wall second, and `07:05:52Z` was a local time with a UTC suffix bolted on. Both were acted on by someone else before being caught — one as a 1.78× "unreconciled gap", one as a hung job. `Z` **is the one suffix that asserts a timezone**, so a time taken from a machine's own clock is written in that machine's zone and named, or converted before it is quoted.
- **A counter read from a RUNNING process is a reading of a moment, not a total.** Sample it after the thing finishes, or record the elapsed time beside it so the shortfall is computable later. `rchar` taken 20.5s before an index ended was 4.92 GB light, and that was enough to make a correct model look wrong rather than to look like a bad sample.
- **Before concluding anything from a refusal or a zero, point the instrument at a case you know is positive.** One query that *must* succeed, failing identically, is what proved the registry was answering about credentials. There was no equivalent control for the other four, which is why they survived as long as they did.

This is the same failure as "*a green must come from the thing being tested, not from the tester noticing something*" in `CLAUDE.md`, reached from the measurement side rather than the test side.

Pinning a hash: the fixture has to make every step load-bearing — 2026-11-02

Written after the hash-once change, where the pin on `package_hash` was the entire safety story and **the first version of it could not fail in one of the three ways that mattered.**

`package_hash` is SHA-256 over the sorted `<lowercased name>:<sha256 of content>` lines of every file in a package. The fixture pinned it with five files — `song.ini`, `notes.chart`, `song.ogg`, `album.png`, `extra.bin`. Red-proofing deleted the `strings.ToLower` and **the pin stayed green**, because every name in the fixture was already lowercase, so the step it was meant to guard was a no-op on that input. Adding one `ReadMe.TXT` made it load-bearing and the pin failed as it should.

Three rules fall out of that, and the second is the one nobody writes down:

1. **Write the pin BEFORE the change, and watch it pass on the OLD code.** A pin written afterwards records what the new code does, which is not the question. This one passed on the pre-change implementation first, so "the value did not move" is a measurement.
2. **The fixture must exercise every step of the algorithm, or the pin is partial.** Ask, step by step, what input would make this line matter — lowercasing needs a capital, sorting needs names that are out of order, a separator needs a name containing it. A fixture that is uniform in some dimension cannot see that dimension.
3. **Validate the pinned VALUE against the spec, not against the code that produced it.** An independent implementation of the documented algorithm agreed on `ec39edc6a292076a7b37fff629588000dbba8a98dfa05e799426f635d7996f62`. Without that, a pin only says "the code still does what it did", which is worthless if what it did was wrong.

What the hole was actually worth: 1 uppercase name in 26,575

The real library was then asked, rather than reasoned about — one read-only pass over `/api/v1/songs` across 4,085 songs:

asset names collected	26,575
distinct asset names	**41**
non-ASCII filenames	**0**
filenames containing uppercase	**1** — `album.PNG`
non-ASCII *and* cased	0

So the lowercasing in `package_hash` is load-bearing for exactly **one song in 4,085**. Had the pin stayed unable to fail on it, and had someone later dropped `strings.ToLower`, the blast radius would have been one `package_hash` moving: one `ETag`, one silent re-download for every client, no error, no 500, no log line. **It was not just a test that could not fail — it was a test that could not fail about a defect that could not be seen.** The second is worse, and it is the case regression tests exist for.

Unicode case folding is therefore untested by any fixture, and **deliberately stays that way**: a non-ASCII *filename* is exactly the fixture `CLAUDE.md` forbids, because macOS normalises filenames to NFD and `macos-verify.yml` fails when macOS and Linux disagree hash for hash. The folding is covered instead by feeding a non-ASCII name as a Go **string literal** through `putBytes`, which never touches the filesystem — no file, nothing to normalise, and the test asserts the un-folded name hashes differently so its green cannot be vacuous.

One more thing that pass turned up, worth having for the differential-index design: **41 distinct asset names across the whole library.** The name space is tiny and highly repetitive, so keying a cache on names is cheap and the per-file digests are the entire cost.

And the equivalence check that a single fixture cannot replace: the old binary and the new one were both run over the whole 26-case corpus — archives, `.sng`, `.yargsong`, broken metadata, the bare CR — and **0 of 26** `package_hash` **values moved**. One fixture agreeing proves much less than twenty-six disagreeing would disprove.
